LESSON 7: API & DATA HANDLING

React Client-Server Communication, Network Protocols, State Lifecycles & Production CRUD Systems

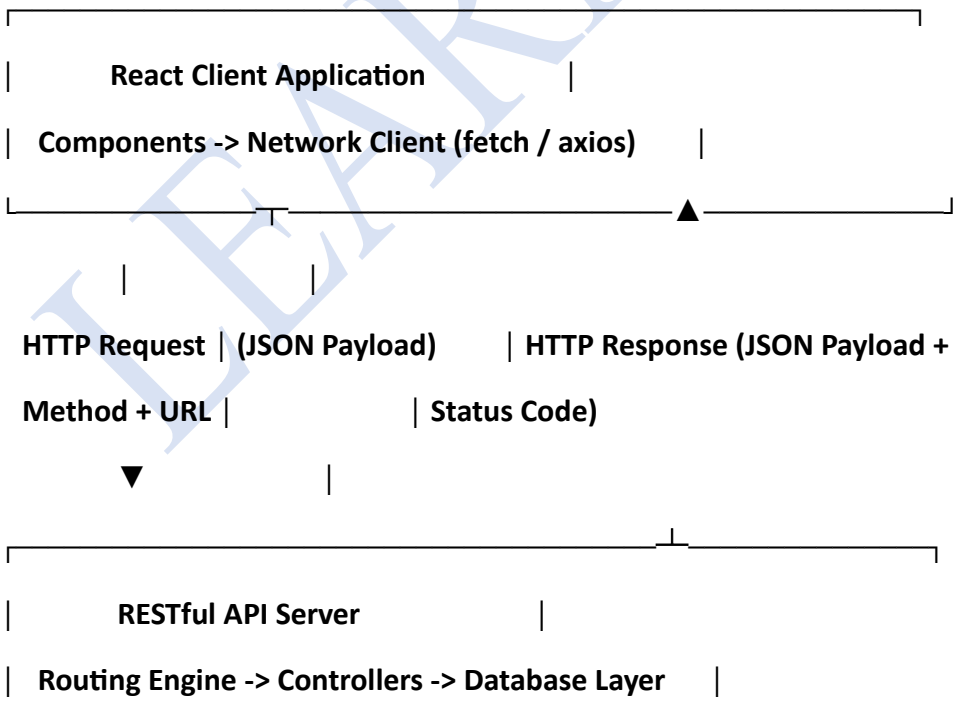
1. REST API FUNDAMENTALS & ARCHITECTURAL OVERVIEW

A modern React frontend is essentially a client-side user interface layer ($UI = f(State)$) decoupled from data persistence. It communicates across network boundaries with a backend application server using an API (Application Programming Interface).

1.1 What is a REST API?

REST (Representational State Transfer) is a software architectural style for distributed hypermedia systems. In REST:

- **Statelessness: Every HTTP request from the client contains all the contextual information and authorization credentials the server needs. No client session state is retained inside the server memory.**
- **Resource-Oriented URIs: Entities (such as users, orders, courses) are modeled as identifiable endpoints using deterministic paths (e.g., /api/v1/courses).**
- **Standard HTTP Methods: Interactions perform well-defined semantics mapped directly to database CRUD operations.**



2. HTTP METHODS & CRUD MAPPING

The foundation of REST lies in standardizing operations across a predictable verb-noun paradigm.

HTTP Method	CRUD Equivalent	Request Body Allowed?	Idempotent?	Typical Target URI	Semantic Function
GET	Read	No	Yes	/api/v1/courses or .../courses/101	Retrieves records without modifying backend data.
POST	Create	Yes (JSON/FormData)	No	/api/v1/courses	Creates a brand-new record; generates a new resource identifier.
PUT	Update (Complete)	Yes	Yes	/api/v1/courses/101	Completely replaces the target entity with the supplied payload.

HTTP Method	CRUD Equivalent	Request Body Allowed?	Idempotent?	Typical Target URI	Semantic Function
PATCH	Update (Partial)	Yes	No / Contextual	/api/v1/courses/101	Modifies only the explicitly provided fields of the target record.
DELETE	Delete	Optional (Rare)	Yes	/api/v1/courses/101	Permanently removes the target record from the database.

The Concept of Idempotency

An HTTP method is idempotent if making multiple identical requests produces the exact same backend state as making a single request:

- **GET is idempotent: Reading an endpoint 10 times does not alter database state.**
- **PUT is idempotent: Overwriting an entity with { "title": "React" } 10 times leaves the entity in the exact same state.**
- **DELETE is idempotent: Deleting ID 101 leaves it deleted whether requested once or ten times (even if subsequent HTTP status codes return 404 Not Found).**
- **POST is NOT idempotent: Dispatching a POST request 5 times creates 5 distinct database records with 5 unique primary keys.**

3. HTTP STATUS CODES THAT EVERY REACT DEVELOPER MUST KNOW

Handling API responses requires inspecting the HTTP status code returned by the server before parsing payloads.

1xx: Informational (Request received, continuing process)

2xx: Success (The action was successfully received and accepted)

3xx: Redirection (Further action must be taken to complete request)

4xx: Client Error (The request contains bad syntax or cannot be fulfilled)

5xx: Server Error (The server failed to fulfill an apparently valid request)

Essential Status Codes in React State Management:

- **200 OK: Successful GET, PUT, or PATCH. Body contains data payload.**
- **201 Created: Successful POST. The record was inserted; often returns the created record with its assigned id.**
- **204 No Content: Successful DELETE or PUT. The server processed the request, but intentionally returns no body payload.**
- **400 Bad Request: Client-side validation failure (e.g., missing required fields, invalid email format).**
- **401 Unauthorized: User is unauthenticated. Token is missing, invalid, or expired (requires login redirect).**
- **403 Forbidden: Authenticated, but lacking permissions (e.g., standard user attempting an admin delete).**
- **404 Not Found: Target endpoint or resource ID does not exist.**
- **409 Conflict: Duplicate resource conflict (e.g., user registration with an email that already exists).**
- **500 Internal Server Error: Unhandled exception or database crash on the backend.**

4. fetch() VS axios: THE DEFINITIVE ARCHITECTURAL COMPARISON

React provides native access to the browser's fetch() API, but enterprise applications often use third-party HTTP clients like Axios.

4.1 Native Browser fetch() API

window.fetch() is built into all modern browsers. It relies entirely on Promises.

The Critical fetch Error Catch:

fetch() does not reject a promise on HTTP errors (like 404 or 500). A fetch() promise rejects *only* on network failure or if the request is blocked (e.g., DNS failure, offline client, CORS block). To catch 4xx/5xx errors, developers must manually verify response.ok:

JavaScript

```
// Native fetch requires explicit status checking and two-step JSON deserialization
const loadData = async () => {
 try {
  const res = await fetch("https://api.example.com/items");

  // Manual HTTP error handling required:
  if (!res.ok) {
   throw new Error(`HTTP Error status: ${res.status} ${res.statusText}`);
  }

  // Manual JSON body parsing step:
  const data = await res.json();
  return data;
 } catch (err) {
  console.error("Network or parsed error:", err.message);
 }
};
```

4.2 Axios HTTP Library

Axios is an external, promise-based HTTP client library installable via npm install axios.

Why Production Teams Choose Axios:

1. **Automatic JSON Transformation: Serializes outgoing JavaScript objects to JSON strings and parses incoming JSON responses automatically. Data is directly accessible via response.data.**

2. **Automatic HTTP Error Rejection: Automatically rejects the promise if the server returns any status outside the 2xx range, routing execution straight to the .catch() block.**
3. **Request/Response Interceptors: Allows developers to intercept requests globally (e.g., injecting an Authorization: Bearer <token> header on every outgoing call) and responses (e.g., catching 401 errors globally to refresh auth tokens or redirect to login).**
4. **Built-in Request Cancellation: Clean timeout support and native integration with AbortController.**

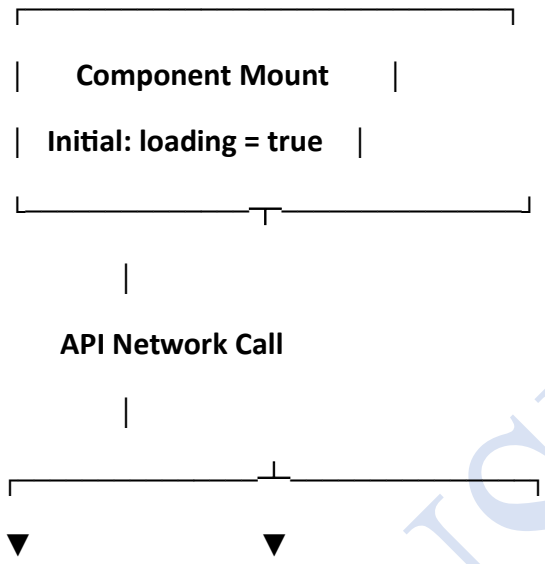
Architectural Feature Matrix

Functional Requirement	Native fetch()	Third-Party axios
Installation	Built-in (Zero bundle size impact)	External library (npm i axios, ~13kB)
Response Body Parsing	Two-step process (await res.json())	Automatic (Directly mapped to res.data)
4xx / 5xx HTTP Errors	Resolves normally (Must check res.ok)	Rejects Promise automatically
Payload Serialization	Manual (body: JSON.stringify(data))	Automatic (axios.post(url, data))
Request Interceptors	No native support (Requires wrappers)	First-class native support (axios.interceptors)
Timeout Configuration	Requires manual AbortController setup	Native configuration flag (timeout: 5000)

5. UI STATE MANAGEMENT: THE "LOADING-DATA-ERROR" TRIAD

Network operations are asynchronous. They take an indeterminate amount of time (milliseconds to seconds), and can fail due to poor connectivity, dropped connections, or server crashes.

To maintain a responsive, bug-free interface, every data-driven React component must account for the Three Pillars of Async UI:



5.1 Anti-Pattern: Primitive Boolean Blindness

Storing multiple independent booleans can result in impossible UI states if developers forget to synchronize them:

JavaScript

```
// ❌ RISKY PATTERN: Possible to have isLoading=true AND isError=true simultaneously!
const [isLoading, setIsLoading] = useState(false);
const [isError, setIsError] = useState(false);
const [isSuccess, setIsSuccess] = useState(false);
```

5.2 Best Practice: State Machine / Dedicated Async Boundaries

Ensure your state transition cleanly resets previous error states when re-initiating requests:

JavaScript

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);


const executeFetch = async () => {
  setLoading(true);
  setError(null); // Reset prior errors before attempting!
  try {
    const res = await fetch(ENDPOINT);
    if (!res.ok) throw new Error(`Fetch failure: ${res.status}`);
    const result = await res.json();
    setData(result);
  } catch (err) {
    setError(err.message || "An unexpected error occurred.");
  } finally {
    setLoading(false); // Guarantees loading is cleared in both success and failure paths
  }
};
```

6. COMPLETE PRODUCTION CRUD IMPLEMENTATION (ENTERPRISE GRADE)

Below is a production-quality, end-to-end Course Management dashboard. It illustrates:

- **Full CRUD flow (GET, POST, PUT/PATCH, DELETE).**
- **Synchronized Loading, Error, and Validation states.**
- **Modal-less, in-place edit modes.**
- **Defensive programming with try / catch / finally.**

- Clean functional state updates ensuring zero UI race conditions.

JavaScript

```
import React, { useState, useEffect } from "react";


const API_BASE = "https://jsonplaceholder.typicode.com/posts";


export default function CourseCrudMaster() {
  // 1. ASYNC CORE STATES
  const [courses, setCourses] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [errorMessage, setErrorMessage] = useState("");
  const [actionInProgress, setActionInProgress] = useState(false);


  // 2. FORM STATES (Create & Edit Workflows)
  const [formData, setFormData] = useState({ title: "", body: "" });
  const [editingId, setEditingId] = useState(null);


  // ================================================================
  // READ (GET) - Load initial dataset on component mount
  // ================================================================
  const fetchCourses = async () => {
    setIsLoading(true);
    setErrorMessage("");
    try {
      const response = await fetch(`${API_BASE}?_limit=6`);
      if (!response.ok) {
```

```
      throw new Error(`Server responded with status: ${response.status}`);
    }
    const data = await response.json();
    setCourses(data);
  } catch (err) {
    setErrorMessage(`Failed to load courses: ${err.message}`);
  } finally {
    setIsLoading(false);
  }
};

useEffect(() => {
  fetchCourses();
}, []);

// ========================================================================
// CREATE (POST) - Persist new entity to server and sync state
// ========================================================================
const handleCreateCourse = async (e) => {
  e.preventDefault();
  if (!formData.title.trim() || !formData.body.trim()) {
    alert("All fields are required.");
    return;
  }

  setActionInProgress(true);
```

```
setErrorMessage("");
try {
  const response = await fetch(API_BASE, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      title: formData.title,
      body: formData.body,
      userId: 1,
    }),
  });

  if (!response.ok) throw new Error(`POST failed: ${response.status}`);
  const createdItem = await response.json();

  // Ensure stable client-side identification
  const finalizedItem = {
    ...createdItem,
    id: createdItem.id || Date.now(),
  };

  // Immutable state update: prepend new record
  setCourses((prev) => [finalizedItem, ...prev]);
  setFormData({ title: "", body: "" });
} catch (err) {
  setErrorMessage(`Could not create course: ${err.message}`);
```

```
  } finally {
    setActionInProgress(false);
  }
};

// ============================================================================
// UPDATE (PUT/PATCH) - Modify existing entity
// ============================================================================
const handleStartEdit = (course) => {
  setEditingId(course.id);
  setFormData({ title: course.title, body: course.body });
};

const handleCancelEdit = () => {
  setEditingId(null);
  setFormData({ title: "", body: "" });
};

const handleSaveUpdate = async (e) => {
  e.preventDefault();
  setActionInProgress(true);
  setErrorMessage("");

  try {
    const response = await fetch(`${API_BASE}/${editingId}`, {
      method: "PATCH", // Using PATCH for partial field updates
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        title: formData.title,
        body: formData.body,
      }),
    });

    if (!response.ok) throw new Error(`Update failed: ${response.status}`);
    const updatedData = await response.json();

    // Immutable update using Array.prototype.map
    setCourses((prev) =>
      prev.map((item) =>
        item.id === editingId
          ? { ...item, title: updatedData.title, body: updatedData.body }
          : item
      )
    );

    handleCancelEdit();
  } catch (err) {
    setErrorMessage(`Update operation failed: ${err.message}`);
  } finally {
    setActionInProgress(false);
  }
};
```

```
// ========================================================================
// DELETE - Remove entity from remote server and local state
// ========================================================================
const handleDeleteCourse = async (targetId) => {
 const confirmDelete = window.confirm(
  "Are you sure you want to permanently delete this course?"
 );
 if (!confirmDelete) return;

 setActionInProgress(true);
 setErrorMessage("");
 try {
  const response = await fetch(`${API_BASE}/${targetId}`, {
   method: "DELETE",
  });

  if (!response.ok) throw new Error(`Deletion failed: ${response.status}`);

  // Immutable deletion using Array.prototype.filter
  setCourses((prev) => prev.filter((item) => item.id !== targetId));
 } catch (err) {
  setErrorMessage(`Delete failed: ${err.message}`);
 } finally {
  setActionInProgress(false);
 }
```

```
};

// ========================================================================
// RENDER PIPELINE: CONDITIONAL UI CHECKS
// ========================================================================
return (
  <div style={{ maxWidth: 800, margin: "20px auto", fontFamily: "Segoe UI, sans-serif" }}>
    <header>
      <h2>Curriculum & Course Management Portal</h2>
      <p>Enterprise RESTful Integration Showcase</p>
    </header>

    {/* Global Error Notification Banner */}
    {errorMessage && (
      <div style={{ padding: 12, background: "#fee2e2", color: "#b91c1c", borderRadius: 6, marginBottom: 16 }}>
        <strong>System Alert:</strong> {errorMessage}
      </div>
    )}

    {/* Dual Purpose Form (Create / Update) */}
    <section style={{ background: "#f8fafc", padding: 20, borderRadius: 8, border: "1px solid #e2e8f0" }}>
      <h3>{editingId ? "Update Course Record" : "Add New Course Entity"}</h3>
      <form onSubmit={editingId ? handleSaveUpdate : handleCreateCourse}>
        <div style={{ marginBottom: 12 }}>
```

```
        <label style={{ display: "block", marginBottom: 4, fontWeight: "bold" }}>Course
Title</label>
        <input
          type="text"
          value={formData.title}
          onChange={(e) => setFormData({ ...formData, title: e.target.value })}
          placeholder="e.g., Advanced Distributed Systems"
          style={{ width: "100%", padding: 8, boxSizing: "border-box" }}
          disabled={actionInProgress}
        />
      </div>

      <div style={{ marginBottom: 12 }}>
        <label style={{ display: "block", marginBottom: 4, fontWeight: "bold" }}>Syllabus
Description</label>
        <textarea
          value={formData.body}
          onChange={(e) => setFormData({ ...formData, body: e.target.value })}
          placeholder="Provide a comprehensive breakdown of objectives..."
          rows={3}
          style={{ width: "100%", padding: 8, boxSizing: "border-box" }}
          disabled={actionInProgress}
        />
      </div>

      <div style={{ display: "flex", gap: 10 }}>
        <button
```

```
          type="submit"
          disabled={actionInProgress}
          style={{
            background: editingId ? "#0284c7" : "#16a34a",
            color: "#fff",
            border: "none",
            padding: "8px 16px",
            borderRadius: 4,
            cursor: actionInProgress ? "not-allowed" : "pointer",
          }}
        >
          {actionInProgress
            ? "Processing..."
            : editingId
            ? "Save Updates"
            : "Create Course"}
        </button>

        {editingId && (
          <button
            type="button"
            onClick={handleCancelEdit}
            disabled={actionInProgress}
            style={{ background: "#64748b", color: "#fff", border: "none", padding: "8px 16px",
borderRadius: 4 }}
          >
```

```
            Cancel
          </button>
        )}
      </div>
    </form>
  </section>

  {/* Primary Data Display Layer */}
  <section style={{ marginTop: 30 }}>
    <h3>Published Course Catalog</h3>

    {/* Loading Skeleton / Text */}
    {isLoading && <p>Connecting to backend API, retrieving catalog...</p>}

    {/* Empty Collection State */}
    {!isLoading && courses.length === 0 && (
      <p style={{ fontStyle: "italic" }}>The catalog is currently empty. Add a new course above.</p>
    )}

    {/* Populated List */}
    {!isLoading && courses.length > 0 && (
      <div style={{ display: "grid", gap: 12 }}>
        {courses.map((course) => (
          <article
            key={course.id}
```

```
          style={{
            padding: 16,
            borderRadius: 6,
            border: "1px solid #cbd5e1",
            background: "#ffffff",
            display: "flex",
            justifyContent: "space-between",
            alignItems: "flex-start",
          }}
        >
          <div style={{ maxWidth: "75%" }}>
            <h4 style={{ margin: "0 0 8px 0", color: "#0f172a" }}>{course.title}</h4>
            <p style={{ margin: 0, color: "#475569", fontSize: 14 }}>{course.body}</p>
          </div>

          <div style={{ display: "flex", gap: 8 }}>
            <button
              onClick={() => handleStartEdit(course)}
              disabled={actionInProgress}
              style={{ background: "#e0f2fe", color: "#0369a1", border: "none", padding: "6px
12px", borderRadius: 4, cursor: "pointer" }}
            >
              Edit
            </button>

            <button
```

```
                onClick={() => handleDeleteCourse(course.id)}
                disabled={actionInProgress}
                style={{ background: "#fee2e2", color: "#b91c1c", border: "none", padding: "6px 12px", borderRadius: 4, cursor: "pointer" }}
              >
                Delete
              </button>
            </div>
          </article>
        ))}
      </div>
    )}
  </section>
 </div>
);
}
```

7. ADVANCED ARCHITECTURAL PATTERN: OPTIMISTIC UI UPDATES

In a traditional (pessimistic) UI update, the application displays a loading spinner and blocks interface changes until the remote server finishes processing and returns an HTTP 200/201.

What is Optimistic UI?

Optimistic UI assumes the server operation will succeed 99% of the time. The interface updates the local React state immediately, before dispatching the background network call.

- **If the server succeeds: The background request quietly completes; the user experiences a zero-latency interface.**
- **If the server fails: The component catches the rejection, alerts the user, and rolls the state back to its previous snapshot.**

Implementation: Optimistic Delete Pattern

JavaScript

```
const deleteCourseOptimistic = async (targetId) => {
  // 1. Take a snapshot of the current state before modifying
  const previousCourses = [...courses];

  // 2. OPTIMISTICALLY update client UI instantly
  setCourses((prev) => prev.filter((course) => course.id !== targetId));

  try {
    const res = await fetch(`https://api.example.com/courses/${targetId}`, {
      method: "DELETE",
    });

    if (!res.ok) {
      throw new Error("Server rejected delete request.");
    }
    // Success: State is already correct, no action required.
  } catch (err) {
    // 3. ROLLBACK: Revert to previous snapshot on network or server error
    alert(`Could not delete item: ${err.message}. Reverting UI.`);
    setCourses(previousCourses);
  }
};
```

8. PRODUCTION AXIOS ARCHITECTURE: CENTRALIZED API CLIENT

Scattering raw fetch() or axios.get() calls with hardcoded URLs across tens of components introduces maintenance debt. Production code isolates networking inside a centralized API service layer.

8.1 Setting Up a Custom Axios Client (apiClient.js)

JavaScript

```
import axios from "axios";

const apiClient = axios.create({
  baseURL: process.env.REACT_APP_API_BASE_URL || "https://api.example.com/v1",
  timeout: 10000, // Abort request if it exceeds 10 seconds
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
});

// Request Interceptor: Automatically inject authorization bearer tokens
apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("auth_token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response Interceptor: Globally intercept 401s and format standard error messages
```

```
apiClient.interceptors.response.use(
  (response) => response.data, // Strip axios metadata; return parsed JSON body directly
  (error) => {
    if (error.response && error.response.status === 401) {
      console.warn("Session expired. Forcing re-authentication.");
      window.location.href = "/login";
    }
    const customError = error.response?.data?.message || error.message || "Network communication failed";
    return Promise.reject(new Error(customError));
  }
);

export default apiClient;
```

8.2 Isolating Domain Operations (courseService.js)

JavaScript

```
import apiClient from "./apiClient";

export const CourseService = {
  getAll: (limit = 10) => apiClient.get(`/courses?limit=${limit}`),
  getById: (id) => apiClient.get(`/courses/${id}`),
  create: (payload) => apiClient.post("/courses", payload),
  update: (id, payload) => apiClient.patch(`/courses/${id}`, payload),
  delete: (id) => apiClient.delete(`/courses/${id}`),
};
```

9. TOP ANTI-PATTERNS IN REACT DATA HANDLING

Anti-Pattern 1: The Missing Dependency Infinite Render Loop

JavaScript

```
// ❌ CRITICAL BUG: Runs on every render, sets state, triggers re-render forever!
useEffect(() => {
  fetchData();
}); // Missing dependency array!
```

- **Why it happens: Omitting the second argument ([]) tells React to run the effect after *every single render pass*. Calling setState inside the effect triggers an immediate re-render, locking the browser thread into an infinite loop.**
- **Fix: Provide an empty array [] if the call should only run on mount, or explicitly list the trigger dependencies (e.g., [courseId]).**

Anti-Pattern 2: Unhandled Unmounted Component State Updates

When a user navigates away from a page while a slow API request is in flight, the promise resolves *after* the component has unmounted. Calling setState on an unmounted component wastes memory and can leak resources.

Fix: Aborting In-Flight Requests via AbortController

JavaScript

```
useEffect(() => {
  const controller = new AbortController();
  const { signal } = controller;

  const loadData = async () => {
    try {
      const res = await fetch(API_URL, { signal });
      const data = await res.json();
      setCourses(data);
    } catch (err) {
```

```
    if (err.name !== "AbortError") {
      setError(err.message);
    }
  }
 };

 loadData();

 // Cleanup runs on unmount: Cancels network transmission immediately!
 return () => {
  controller.abort();
 };
}, []);
```

Anti-Pattern 3: In-Place Mutation of State Arrays

```
JavaScript
// ❌ WRONG: Pushing into state array directly
courses.push(newCourse);
setCourses(courses); // Memory pointer identical! React skips re-render!

// ✅ CORRECT: Immutable transformation
setCourses((prev) => [...prev, newCourse]);
```

10. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Frequency Interview Questions

Q1: Why doesn't fetch() reject a promise when the server returns a 404 or 500 error?

Answer: **The architecture of the browser's fetch() API treats an HTTP response code—even an error status like 404 Not Found or 500 Internal Server Error—as a successfully completed**

HTTP transaction. The browser successfully made contact with the server and received a valid HTTP response. The promise only rejects if there is an actual network failure, a blocked domain, or a CORS error preventing the browser from receiving the response. Developers must check response.ok (which validates that status is within 200-299).

Q2: What is the semantic and functional difference between PUT and PATCH?

Answer: **Both update existing resources. However, PUT is designed to completely replace the target resource; any omitted attributes are overwritten or nullified according to REST conventions. PATCH is designed for partial updates, modifying only the fields explicitly supplied in the request body while leaving all other fields intact.**

Q3: How does Axios handle error responses differently than native fetch()?

Answer: **Axios automatically checks the status code of incoming HTTP responses. If the status falls outside the 2xx range, Axios rejects the promise and creates an error object containing the server's response (error.response), sending execution straight to the .catch() block.**

Q4: Explain the importance of the finally block in asynchronous React data fetching.

Answer: **The finally block runs regardless of whether the try block succeeds or the catch block intercepts an error. In React, this is typically used to reset the loading indicator (setIsLoading(false)), ensuring that spinners do not get stuck on screen when network calls fail.**

Q5: What is an Optimistic UI update and when should it be avoided?

Answer: **Optimistic UI updates the client-side state immediately under the assumption that the backend request will succeed, reverting the state back if the server returns an error. It provides a near-zero latency experience for low-risk actions like "Likes", "Bookmarks", or "Deletions". It should be avoided for high-stakes, validation-heavy operations, such as financial checkouts, multi-step loan applications, or password changes.**